Session 14: Advanced NumPy - Detailed Lecture Notes

1. NumPy vs. Python Lists

The teacher begins by explaining why NumPy was created. Python's built-in data types (like lists) are very slow for large datasets. NumPy is preferred in Data Science for three main reasons: **Speed, Memory, and Convenience.**

A. Speed of Execution

NumPy is significantly faster than Python lists because it uses C-style arrays (static, not referential).

Example Code (Python List):

```
import time

# Create 2 lists with 1 crore (10 million) items each
a = [i for i in range(10000000)]
b = [i for i in range(10000000)]

c = []
start = time.time()
for i in range(len(a)):
    c.append(a[i] + b[i])
print("Python List Time:", time.time() - start)
```

Line-by-line explanation:

1. import time: Imports the time module to measure execution speed.
2. a = [...] and b = [...]: Creates two lists using list comprehension, each containing 10 million integers.
3. c = []: Initializes an empty list to store the results.
4. start = time.time(): Records the current time before the loop starts.
5. for i in range(len(a)): Loops through every index from 0 to 9,999,999.
6. c.append(a[i] + b[i]): Adds the items at the same index and appends to c.
7. time.time() - start: Subtracts start time from end time to find total duration.

Example Code (NumPy):

```
import numpy as np
```

```
a = np.arange(10000000)
b = np.arange(10000000)

start = time.time()
c = a + b
print("NumPy Time:", time.time() - start)
```

Line-by-line explanation:

1. import numpy as np: Imports the NumPy library.
2. a = np.arange(10000000): Creates a NumPy array with 10 million items.
3. c = a + b: Performs vectorized addition (no loop needed).

Why is NumPy faster?

- **C-type Arrays:** It stores items directly in memory, not addresses.
- **No Referencing:** Python lists store addresses of objects, requiring an extra step to find the value.
- **Fixed Size:** Unlike lists, which double in size and copy data when full, NumPy arrays are static.

B. Memory Consumption

NumPy allows you to define specific data types (like int8, int32), which can save massive amounts of RAM.

Key Point: Python integers are objects and take about 28 bytes each. A NumPy int32 takes only 4 bytes.

C. Convenience

As seen in the code above, adding two arrays in NumPy requires a simple a + b, whereas Python lists require a for loop and append logic.

2. Advanced Indexing

The teacher reviews basic indexing (row, column) and introduces two advanced techniques.

A. Fancy Indexing

Used when you want to fetch multiple specific rows or columns that do not follow a simple slice pattern (e.g., Row 0, 2, and 3).

Code:

```
a = np.arange(24).reshape(6,4)

# Fetching Row 0, 2, and 5
print(a[[0, 2, 5]])
```

Line-by-line explanation:

1. a = np.arange(24).reshape(6,4): Creates a 2D array of 6 rows and 4 columns.
2. a[[0, 2, 5]]: Instead of a single index, we pass a **list** of indices. This tells NumPy to return the rows at those specific positions.

B. Boolean Indexing (Boolean Masking)

This is used to filter data based on conditions. It is one of the most important concepts for Data Analysis.

Code:

```
a = np.random.randint(1, 100, 24).reshape(6, 4)

# 1. Find all numbers > 50
print(a[a > 50])

# 2. Find even numbers
print(a[a % 2 == 0])

# 3. Multiple conditions (Greater than 50 AND Even)
print(a[(a > 50) & (a % 2 == 0)])

# 4. Find numbers NOT divisible by 7
print(a[~(a % 7 == 0)])
```

Line-by-line explanation:

1. a > 50: This creates a "Boolean Mask" (an array of True/False values).
2. a[a > 50]: By passing the mask back into a[], NumPy only returns the items where the condition was True.
3. &: Used for bitwise 'AND'.
4. ~: The "Not" operator (Tilde) used to invert the boolean mask.

3. Broadcasting

Definition: Broadcasting describes how NumPy treats arrays with different shapes during

arithmetic operations.

The Rules of Broadcasting

To perform an operation between Array A and Array B:

1. **Rule 1:** If the arrays have a different number of dimensions, prepend the shape of the smaller array with 1s on its left side until both have the same number of dimensions.
2. **Rule 2:** If the size of a dimension does not match, the array with size 1 in that dimension is stretched to match the other array's size.
3. **Rule 3:** If in any dimension the sizes disagree and neither is equal to 1, an error is raised.

Example 1 (Works):

- Array A: (3, 3)
- Array B: (3,) -> becomes (1, 3) (Rule 1) -> becomes (3, 3) (Rule 2).
- Result: Operation is possible.

Example 2 (Fails):

- Array A: (3, 2)
- Array B: (3, 3)
- Result: Disagreeing dimensions and neither is 1. **ValueError**.

Why use it? It enables **Vectorization**, allowing NumPy to avoid slow Python loops and perform calculations at C-speed.

4. Working with Mathematical Formulas

NumPy allows you to apply any complex mathematical formula to an entire array at once.

Example: Sigmoid Function

Formula: $S(x) = \frac{1}{1+e^{-x}}$

Code:

```
def sigmoid(array):
   return 1 / (1 + np.exp(-array))

a = np.arange(10)
print(sigmoid(a))
```

Explanation:

1. np.exp(-array): Calculates the exponent for every item in the array simultaneously.
2. The entire formula is applied to every element, returning a new array of Sigmoid values.

Example: Mean Squared Error (MSE)

Used to calculate the error between actual values and predicted values.

Formula: $MSE = \frac{1}{n}\sum(Actual - Predicted)^2$

Code:

```
actual = np.random.randint(1, 50, 25)
predicted = np.random.randint(1, 50, 25)

def mse(actual, predicted):
   return np.mean((actual - predicted)**2)

print(mse(actual, predicted))
```

Line-by-line explanation:

1. actual - predicted: Finds the difference (error) for every pair.
2. **2: Squares every difference to remove negative values.
3. np.mean(...): Sums all squared differences and divides by the number of items.

5. Handling Missing Values (NaN)

Missing data in NumPy is represented by np.nan (Not a Number).

Code:

```
a = np.array([1, 2, 3, np.nan, 5])

# Filter out missing values
print(a[~np.isnan(a)])
```

Explanation:

1. np.isnan(a): Returns True for the missing value.
2. ~: Inverts it so we get True for valid numbers.
3. Result: Returns only [1, 2, 3, 5].

6. Plotting Graphs

You can use NumPy to generate points for any mathematical function and plot them using Matplotlib.

Code (Plotting a 2D Parabola $y = x^2$):

```
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100)
y = x**2

plt.plot(x, y)
plt.show()
```

Line-by-line explanation:

1. np.linspace(-10, 10, 100): Generates 100 equally spaced numbers between -10 and 10.
2. y = x**2: Squares every value of x to get y coordinates.
3. plt.plot(x, y): Connects the points on a graph.

Other plots shown:

- **Straight line:** $y = x$
- **Trig functions:** $y = \sin(x)$
- **Complex functions:** $y = x \log(x)$

Summary Tips:

- Always try to use NumPy built-in functions (like np.sum(), np.mean()) instead of loops.
- Use **Broadcasting** to make your code more efficient.
- Use **Boolean Indexing** to filter datasets quickly without complex if-else logic.